

第9章: スタイリングとUI (NativeWind)

アプリの機能は完成しました。この章では「見た目」をより効率よく整える方法を学びます。本書では **NativeWind** (Tailwind CSSの書き方をReact Nativeで使えるライブラリ) を導入します。まずスタイリング手法の選択肢を比較し、NativeWindを導入して、これまで作った画面をより少ないコードで書き直します。

1. スタイリング手法の選択肢を比較する

第4章では **StyleSheet** を使いました。React Nativeのスタイリングには他にも選択肢があります。「どんな場合にどれを選ぶべきか」を解説します。

1.1 比較表

手法	記述スタイル	記述量	学習コスト	Web版との共通性	向いている人
StyleSheet (RN標準)	オブジェクトで定義	多い	低 (追加不要)	△	追加ライブラリを増やしたくない人
NativeWind (本書採用)	クラス名で指定	少ない	低 (Tailwind経験者は即)	◎	Web/TailwindとUIを共通化したい人
Tamagui	専用コンポーネント	中	中～高	△	高性能・高機能を求める人
UIキット (gluestack / React Native Paper等)	完成部品を使う	少ない	中	△	デザインを自分で考えず素早く作りたい人

1.2 それぞれの解説

◆ StyleSheet (RN標準)

第4章で使った方法。 `StyleSheet.create({ ... })` でスタイルを定義し、名前で参照します。

- **長所:** 追加ライブラリ不要／React Nativeの基本そのもの／どんな教材でも通用する
- **短所:** 記述量が多い／小さな調整でもスタイル定義が増えていく

◆ NativeWind（本書の採用）

Tailwind CSS という「短いクラス名で見た目を指定する」仕組みを、React Nativeで使えるようにしたもの。

`className="text-lg font-bold text-blue-600"` のように書きます。

- **長所:** 記述量が少ない／Web版チュートリアル（Tailwind）と書き方が同じ／デザインの一貫性を保ちやすい
- **短所:** クラス名を覚える必要がある（が、よく使うものは限られる）

◆ Tamagui（タマグイ）

高性能を売りにした高機能スタイリング＋UIライブラリ。アニメーションやテーマ切替に強い。

- **長所:** 高性能／機能豊富
- **短所:** 学習コストが高め／初心者にはオーバースペックなことが多い

◆ UIキット（gluestack-ui / React Native Paper など）

ボタンやカードなどの「完成された部品」が最初から用意されているライブラリ。

- **長所:** デザインを考えなくても整った見た目になる／開発が速い
- **短所:** キット独自の作法を学ぶ／細かいカスタマイズに制約

どんな時にどれを選ぶ？（まとめ） - Web/Tailwindの経験があり、UIを共通化したい → NativeWind（本書） - ライブラリを増やさず基本に忠実でいたい → StyleSheet - 凝ったアニメーションや高性能が必要 → Tamagui - デザインを自分で作らず最速で形にしたい → UIキット

2. NativeWind のセットアップ

2.1 インストール

第1章で作った `my-books-app` フォルダで、次を順に実行します。

```
npx expo install nativewind tailwindcss react-native-reanimated react-native-safe-area-context
# nativewind           : 本体 (TailwindをRNで使えるようにする)
# tailwindcss           : Tailwindのクラス定義の土台
# react-native-reanimated : NativeWindが内部で利用するアニメーション部品
# react-native-safe-area-context: 安全領域の部品 (第4章で登場。NativeWindと相性よく使う)
```

2.2 Tailwind設定ファイルを作る

次のコマンドで、Tailwindの設定ファイルのひな形を生成します。

```
npx tailwindcss init
# tailwindcss init : tailwind.config.js という設定ファイルを生成するコマンド
```

生成された `tailwind.config.js` を次のように書き換えます。

▼ このコードがやること（先に日本語で）：NativeWind（Tailwind）に「クラス名をどのファイルから探すか」と「NativeWind用の設定一式を読み込む」ことを教える設定ファイルです。一番大事なのは `content` ——ここに書いたファイルの中で実際に使われているクラスだけがスタイルとして取り込まれます。書き換える箇所は数行だけで、残りはほぼ決まり文句（おまじない）です。詳しい意味は各行のコメントを見てください。

```
// tailwind.config.js - Tailwind/NativeWindの設定

/** @type {import('tailwindcss').Config} */
module.exports = {
  // content : Tailwindのクラスを「どのファイルから探すか」を指定する
  // app配下とcomponents配下の .tsx などを対象にする
  content: ["../app/**/*.{js,jsx,ts,tsx}", "./components/**/*.{js,jsx,ts,tsx}"],
  // presets : NativeWind用の設定一式を読み込む（おまじない）
  presets: [require("nativewind/preset")],
  theme: {
    extend: {}, // ここに独自の色やサイズを追加できる（今は空でOK）
  },
  plugins: [],
};
```

content の指定が重要: Tailwindは「実際に使われているクラスだけ」を最終的に取り込みます。`content` で指定したファイルからクラス名を探すので、ここにファイルの場所を正しく書かないとスタイルが効きません。`./app/**/*.{...}` は「appフォルダ以下の全階層（**）の、指定拡張子のファイル」という意味です。

▼ コードを1つずつ分解して解説

解説1: 設定全体の入れ物（`module.exports`）

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  // ...（中身は下で順に解説）
};
```

- `module.exports = { ... }` は「このファイルが外に渡す内容」を指定する書き方です。`tailwind.config.js` 全体が「1つの設定オブジェクト（`{ }`）」を外に公開しています。

- 1行目の `/** @type ... */` はコメントで、エディタに「これはTailwindの設定オブジェクトですよ」と教える目印です。書いておくと入力補完が効きますが、消しても動作は変わりません。
- 設定オブジェクトの中に `content` / `presets` / `theme` / `plugins` という4つの項目を並べていきます。

用語: `module.exports` ... JavaScriptのファイル（モジュール）が「自分の中身を他のファイルに渡す」ための仕組み。ここに代入したものが、他の場所から読み込めるようになります。

解説2: クラスを探す場所（`content`）

```
content: ["/app/**/*.{js,jsx,ts,tsx}", "/components/**/*.{js,jsx,ts,tsx}"],
```

- `content` は「`className` をどのファイルから探すか」のリストです。ここに書いたファイルの中で実際に使われているクラスだけが、最終的なスタイルに取り込まれます。
- `./app/**/*.{js,jsx,ts,tsx}` は「`app` フォルダ以下の全階層（`**`）にある、拡張子が `js` / `jsx` / `ts` / `tsx` のファイル」という意味です。`components` フォルダも同様に対象にしています。
- ここを書き忘れたり場所を間違えると、クラスが見つからずスタイルが何も効かなくなるので、最重要の項目です。

用語: グロブ（glob） ... `**` や `*` を使ってファイルの場所をまとめて指定する書き方。`**` は「何階層でも下までたどる」、`*` は「任意のファイル名」を表します。

解説3: NativeWind用設定と拡張枠（`presets` / `theme` / `plugins`）

```
presets: [require("nativewind/preset")],
theme: {
  extend: {},
},
plugins: [],
```

- `presets` は「あらかじめ用意された設定一式を読み込む」項目です。`require("nativewind/preset")` で NativeWind用の設定をまとめて取り込んでいます（ほぼ決まり文句）。
- `theme.extend` は「独自の色やサイズを追加する」場所です。今は `{}`（空）でOK。たとえば自社カラーを足したくなったらここに書きます。
- `plugins` は「追加機能（プラグイン）を入れる」リストで、今回は不要なので空配列 `[]` のままにしています。

用語: `require(...)` ... 他のファイルやライブラリの中身を読み込む関数。`module.exports` で公開されたものを、こちらで受け取るイメージです。

2.3 グローバルCSSを作る

`app` フォルダに `global.css` を新規作成し、Tailwindの基本指定を書きます。

▼ このコードがやること（先に日本語で）：Tailwindの土台となるスタイルを読み込むための、3行だけのCSSファイルを作ります。`@tailwind base / components / utilities` の3行を書くことで、`text-lg` や `bg-white` といった便利クラスがアプリで使えるようになります。これはNativeWindを使うための「お決まりの3行」なので、丸ごとそのまま書けばOKです。

```
/* app/global.css - Tailwindの基本スタイルを読み込む */
@tailwind base;          /* Tailwindの土台スタイル */
@tailwind components;    /* コンポーネント系スタイル */
@tailwind utilities;     /* text-lg などの便利クラス群 */
```

▼ コードを1つずつ分解して解説

解説1: 3つの `@tailwind` がそろってTailwindが使えるようになる

```
@tailwind base;          /* Tailwindの土台スタイル */
@tailwind components;    /* コンポーネント系スタイル */
@tailwind utilities;     /* text-lg などの便利クラス群 */
```

- `@tailwind` は「Tailwindの該当部分のスタイルを、ここに展開してね」とCSSに指示する書き方です。3行それぞれが別の役割を持っています。
- `base` ... 文字や余白などの「土台となる初期スタイル」を読み込みます。ブラウザごとの見た目のばらつきをそろえる役目です。
- `components` ... ボタンやカードといった「部品向けのスタイル」を読み込む枠です。
- `utilities` ... `text-lg` や `bg-white` のような「短いクラス名で1つの見た目を当てる便利クラス群」を読み込みます。NativeWindで毎回使うのは主にこれです。
- この3行はNativeWindを使うためのお決まりなので、丸ごとそのまま書けばOKです。

用語: ユーティリティクラス ... `p-4`（余白）や `text-blue-600`（文字色）のように「1つの見た目だけを当てる小さなクラス」のこと。これらを並べて組み合わせるのがTailwind/NativeWindの基本スタイルです。

2.4 Babel と Metro の設定

NativeWindを動かすため、2つの設定ファイルを調整します。内部の仕組みなので「こう書く」と覚えれば十分ですが、それぞれが何をしているのかを知っておくと、エラー時に対処しやすくなります。

そもそも Babel と Metro とは？ あなたが書く `className="text-lg"` のようなコードは、そのままではスマホは理解できません。アプリを動かす前に、**スマホが分かる形へ変換（へんかん）** する道具が必要です。- **Babel**（バベル）... 新しい書き方（TypeScript、JSX、NativeWindのclassName）を、スマホが解釈できる素のJavaScriptへ**翻訳**する道具。- **Metro**（メトロ）... たくさんのファイルや部品を1つにまとめてスマホへ届ける道具（「バンドラ＝束ねるもの」と呼びます。第1章で登場）。

NativeWindは「`className` を本物のスタイルへ変換する」仕組みなので、このBabelとMetroの両方に「NativeWindの変換も通してね」と教える必要があります。それが下の2ファイルの役割です。

`babel.config.js`（無ければ作成）：

▼ このコードがやること（先に日本語で）：Babel（コードをスマホが分かる形へ翻訳する道具）に「`className`をスタイルに変える翻訳ルールも使ってね」と教える設定です。`jsxImportSource: "nativewind"` と `"nativewind/babel"` の2つが、その指定の中心です。仕組みは下のblockquoteで詳しく説明していますが、まずは丸ごとコピーで構いません。詳細は各行のコメントを参照してください。

```
// babel.config.js - コード変換の設定
module.exports = function (api) {
  api.cache(true); // 設定をキャッシュして高速化（おまじない）
  return {
    // presets : 変換ルールの一式。Expo用 + NativeWind用 (jsxImportSource) を指定
    presets: [
      ["babel-preset-expo", { jsxImportSource: "nativewind" }],
      "nativewind/babel",
    ],
  };
};
```

▼ コードを1つずつ分解して解説

解説1: 設定を関数で返す形（`module.exports = function (api)`）

```
module.exports = function (api) {
  api.cache(true); // 設定をキャッシュして高速化（おまじない）
  return {
    // presets: [ ... ]（中身は次で解説）
  };
};
```

- `babel.config.js` は「**設定オブジェクトを返す関数**」を外に公開する形になっています。`function (api) { ... return { ... } }` の `return` した中身がBabelの設定です。

- `api.cache(true)` は「この設定を毎回作り直さず、結果を覚えておく」ための1行です。変換を速くするためのおまじないで、ほぼ必ずこの形で書きます。
- `api` はBabelが渡してくる「設定用の工具箱」で、ここでは `cache` だけ使っています。

用語: キャッシュ（cache） ... 一度作った結果を覚えておき、次回はそれを使い回す仕組み。毎回同じ計算をやり直さずに済むので速くなります。

解説2: 変換ルールの一式（`presets`）

```
presets: [
  ["babel-preset-expo", { jsxImportSource: "nativewind" }],
  "nativewind/babel",
],
```

- `presets` は「どんな変換ルールを使うか」のリストです。上から順に2つ指定しています。
- `["babel-preset-expo", { jsxImportSource: "nativewind" }]` ... Expo標準の変換ルールに、オプション `{ jsxImportSource: "nativewind" }` を渡しています。これが「`<View className="...">` のようなJSXを変換するとき、NativeWindの仕組みを使ってね」という肝心の指定です。
- `"nativewind/babel"` ... `className` を解釈してスタイルに変えるための、NativeWind専用の変換ルールです。
- つまりこの2つで、Babel側に「classNameをスタイルへ翻訳するルール」を組み込んでいます。基本は丸ごとコピーで構いません。

用語: プリセット（preset） ... 「よく使う変換ルールをひとまとめにしたセット」のこと。1つ指定するだけで、必要な変換がまとめて有効になります。

`metro.config.js`（無ければ作成）:

▼ このコードがやること（先に日本語で）: Metro（ファイルを1つにまとめてスマホへ届ける道具）に「NativeWindの機能を足し、先ほど作った `global.css` を読み込ませる」設定です。

`withNativeWind(...)` は「Expoの標準設定を受け取って、NativeWind対応版にして返す」関数だと考えてください。これも基本はコピーでOK。詳しい役割は下のblockquoteと各行のコメントにあります。


```
// metro.config.js - バンドラ(Metro)の設定。global.cssをNativeWindに渡す
const { getDefaultConfig } = require("expo/metro-config");
const { withNativeWind } = require("nativewind/metro");

const config = getDefaultConfig(__dirname); // Expoの標準設定を取得

// withNativeWind : 標準設定にNativeWindの機能を足す。input に先ほどのCSSを指定
module.exports = withNativeWind(config, { input: "./app/global.css" });
```

2つの設定ファイルが何をしているか（要点）:- **babel.config.js** ... **babel-preset-expo**（Expo標準の変換ルール）に、**{ jsxImportSource: "nativewind" }** を渡しています。これは「**JSX（<View className="...">**のような画面の書き方）を変換するとき、NativeWindの仕組みを使ってね」という指定です。続く **"nativewind/babel"** も、classNameを解釈するための変換ルールです。つまりBabel側に「classNameをスタイルに変える翻訳ルール」を追加しています。- **metro.config.js** ... **getDefaultConfig** でExpoの標準設定を取得し、**withNativeWind(config, { input: "./app/global.css" })** で「標準設定にNativeWindの機能を足し、先ほど作った **global.css** を読み込ませる」設定にしています。

withNativeWind は「設定を受け取って、NativeWind対応版にして返す」関数だと考えてください。

難しく見えますが、やっていることは「**BabelとMetroの両方に、NativeWindの変換を組み込む**」だけです。丸ごとコピーで構いませんが、「className がスタイルに化けるのは、この2ファイルのおかげ」と覚えておくと、もしスタイルが効かないとき真っ先にここを見直せます。

▼ コードを1つずつ分解して解説

解説1: 必要な道具を読み込む（2つの **require**）

```
const { getDefaultConfig } = require("expo/metro-config");
const { withNativeWind } = require("nativewind/metro");
```

- ここでは、設定を組み立てるのに必要な2つの関数を読み込んでいます。
- getDefaultConfig** ... Expoが用意している「Metroの標準設定を取得する」関数です。 **expo/metro-config** から取り出しています。
- withNativeWind** ... 標準設定を受け取って「NativeWind対応版に作り変えて返す」関数です。 **nativewind/metro** から取り出しています。
- const { 名前 } = require(...)** は「読み込んだものの中から、必要な関数だけ名前で取り出す」書き方（分割代入）です。

用語: バンドラ (bundler) ... たくさんのファイルや部品を1つにまとめてアプリへ届ける道具。React NativeではMetroがその役割を担います（第1章で登場）。

解説2: 標準設定を取得してNativeWind対応にする

```
const config = getDefaultConfig(__dirname); // Expoの標準設定を取得

module.exports = withNativeWind(config, { input: "./app/global.css" });
```

- `getDefaultConfig(__dirname)` で、まずExpoの標準設定を `config` に受け取ります。 `__dirname` は「この設定ファイルがある場所（フォルダのパス）」を表す決まり文句です。
- `withNativeWind(config, { input: "./app/global.css" })` で、その標準設定にNativeWindの機能を足し、先ほど作った `global.css` を読み込ませています。 `input` に渡したCSSが、アプリ全体のスタイルの土台になります。
- それを `module.exports = ...` で外に公開することで、Metroがこの「NativeWind対応版の設定」を使うようになります。

用語: `__dirname` ... 「今このファイルが置かれているフォルダの場所」を自動で表す特別な変数。設定ファイルでファイルの場所を伝えるときの定番です。

2.5 グローバルCSSを読み込む

`app/_layout.tsx`（第6章で作成）の先頭に、CSSの読み込みを1行足します。

```
// app/_layout.tsx の一番上に追加
import "./global.css"; // NativeWindのスタイルをアプリ全体で有効にする（先頭で読み込む）

import { Stack } from "expo-router";
// （以下は第6章のまま）
```

2.6 設定を反映させる

設定ファイルを変えたときは、開発サーバーをキャッシュを消して再起動します。

```
npx expo start --clear
# --clear : 以前のキャッシュ（一時保存データ）を消して起動する。設定変更を確実に反映させる
```

--clear を付ける理由: Metro（変換ツール）は速度のため変換結果を一時保存します。設定を変えた直後は古い保存が残っていて反映されないことがあるため、`--clear` で消してから起動します。「設定を変えたのに反映されない」ときの定番の対処です。

3. NativeWind の基本的な書き方

NativeWindでは、`style={...}` の代わりに `className="..."` にクラス名を並べて見た目を指定します。

▼ このコードがやること（先に日本語で）：NativeWindの基本である `className="..."` の書き方を、画面中央に文字を表示する小さな例で体験します。ポイントは「`className` の中に、見た目を表す短いクラス名を空白で区切って並べる」こと——例えば `flex-1`（画面いっぱい）、`justify-center`（縦中央）、`text-2xl`（文字大きめ）のようにです。1つ1つのクラスが何を意味するかは、コード内のコメントで確認してください。

```
import { View, Text } from "react-native";

export default function Demo() {
  return (
    // className に空白区切りでクラスを並べる
    <View className="flex-1 justify-center items-center bg-white">
      /* flex-1 → flex:1 / justify-center → 縦中央 / items-center → 横中央 / bg-white → 背景白 */
      <Text className="text-2xl font-bold text-slate-800">📖 書籍管理アプリ</Text>
      /* text-2xl → 文字大きめ / font-bold → 太字 / text-slate-800 → 濃いグレー文字 */
    </View>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: 外側の枠を画面いっぱいに広げ、中身を中央へ（`<View>` の`className`）

```
<View className="flex-1 justify-center items-center bg-white">
```

- `className="..."` の中に、見た目を表す短いクラス名を空白で区切って並べるのがNativeWindの基本です。ここでは4つ並んでいます。
- `flex-1` ... この `<View>` を「使える空間いっぱいに広げる」指定です（`flex: 1` と同じ）。今回は画面全体に広がります。
- `justify-center` ... 中の要素を「縦方向の中央」にそろえます。
- `items-center` ... 中の要素を「横方向の中央」にそろえます。この2つを合わせて「画面のど真ん中」に配置されます。
- `bg-white` ... 背景を白にします（`bg-` は背景色の指定）。

用語: Flexbox（フレックスボックス） ... 要素の並べ方や中央寄せを決める仕組み。**flex-1**（広がる量）、**justify-center**（主軸方向の中央）、**items-center**（交差軸方向の中央）はこの仕組みのクラスです。

解説2: 文字の見た目を整える（`<Text>` のclassName）

```
<Text className="text-2xl font-bold text-slate-800">📖 書籍管理アプリ</Text>
```

- 文字を表示する `<Text>` にも `className` で見た目を当てます。ここでは3つ並んでいます。
- `text-2xl` ... 文字を「**大きめのサイズ**」にします。`text-sm`（小）→ `text-base`（標準）→ `text-lg` / `text-xl` / `text-2xl`（大きく）と段階があります。
- `font-bold` ... 文字を**太字**にします。
- `text-slate-800` ... 文字色を「**濃いグレー**」にします。`text-色名-濃さ` の形で、濃さは50（薄い）～900（濃い）で指定します。

用語: カラースケール ... Tailwind/NativeWindの色は「色名 + 濃さの数字（50～900）」で表します。`slate-800` なら「スレート（青みグレー）の濃いほう」という意味です。

3.1 第4章のStyleSheetとの対応

第4章で書いたスタイルが、NativeWindのクラスにどう対応するかを表で示します。

やりたいこと	StyleSheet	NativeWind クラス
画面いっぱい	<code>flex: 1</code>	<code>flex-1</code>
縦方向中央	<code>justifyContent: "center"</code>	<code>justify-center</code>
横方向中央	<code>alignItems: "center"</code>	<code>items-center</code>
横並び	<code>flexDirection: "row"</code>	<code>flex-row</code>
内側余白16	<code>padding: 16</code>	<code>p-4</code> (4 = 16px)
文字サイズ大	<code>fontSize: 24</code>	<code>text-2xl</code>
太字	<code>fontWeight: "bold"</code>	<code>font-bold</code>
背景白	<code>backgroundColor: "#fff"</code>	<code>bg-white</code>
角丸	<code>borderRadius: 8</code>	<code>rounded-lg</code>
文字色青	<code>color: "#1e40af"</code>	<code>text-blue-800</code>

数字の対応（重要）：Tailwindの余白やサイズの数字は「4倍するとピクセル」が基本です。`p-4` は `padding: 16`、`p-2` は `padding: 8`、`gap-3` は `gap: 12` です。慣れると暗算できます。色は `text-blue-600` のように「色名-濃さ（50～900）」で指定します。

4. 一覧画面をNativeWindで書き直す

第7・8章で `StyleSheet` で書いた一覧画面を、NativeWindで書き直してみます。`StyleSheet.create` のブロックがごっそり消えて、コードが短くなるのを体感してください。

▼このコードがやること（先に日本語で）：これまで `StyleSheet` で書いていた一覧画面を、すべて `className` で書き直した完成形です。注目してほしいのは、ファイル末尾にあった `StyleSheet.create({ ... })` の長いブロックがまるごと消え、見た目の指定が各部品 `className` に移っている点です。検索ボックス・カード・追加ボタンなどの見た目が、短いクラス名だけで表現されています。各クラスの意味やテンプレートリテラルでの色切り替えは、コード内コメントと直後の補足を見てください。

```
// app/index.tsx - NativeWind版 (主要部分のみ)

import { useState, useCallback, useMemo } from "react";
import { View, Text, FlatList, Pressable, TextInput, ActivityIndicator } from "react-native";
import { useRouter, useFocusEffect } from "expo-router";
import { fetchBooks } from "../lib/books";
import { Book } from "../types/book";

export default function HomeScreen() {
  const router = useRouter();
  const [books, setBooks] = useState<Book[]>([]);
  const [loading, setLoading] = useState(true);
  const [keyword, setKeyword] = useState("");

  const load = useCallback(async () => {
    try {
      setLoading(true);
      setBooks(await fetchBooks());
    } finally {
      setLoading(false);
    }
  }, []);

  useFocusEffect(useCallback(() => { load(); }, [load]));

  const filteredBooks = useMemo(() => {
    if (keyword.trim() === "") return books;
    const lower = keyword.toLowerCase();
    return books.filter(
      (b) => b.title.toLowerCase().includes(lower) ||
b.author.toLowerCase().includes(lower)
    );
  }, [books, keyword]);

  if (loading) {
    // className で中央寄せ。StyleSheet不要
    return (
      <View className="flex-1 justify-center items-center">
        <ActivityIndicator size="large" color="#1e40af" />
      </View>
    );
  }

  return (
    <View className="flex-1 bg-slate-50">
      { /* 検索ボックス */ }
      <View className="p-4 pb-0">
        <TextInput

```

```

        className="bg-white border border-slate-200 rounded-lg px-4 py-2.5 text-sm"
        placeholder="🔍 タイトルや著者で検索..."
        value={keyword}
        onChangeText={setKeyword}
    />
</View>

<FlatList
    data={filteredBooks}
    keyExtractor={({item}) => item.id}
    contentContainerClassName="p-4 gap-2.5" // contentContainerStyleのNativeWind版
    ListEmptyComponent={
        <Text className="text-center text-slate-400 mt-10 px-5">
            {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりませ
ん。"}
        </Text>
    }
    renderItem={({ item }) => (
        // カード。borderやrounded、paddingをすべてクラスで指定
        <Pressable
            className="bg-white rounded-xl p-3.5 border border-slate-200"
            onPress={() => router.push(`/books/${item.id}`)}
        >
            <Text className="text-base font-bold text-slate-800">{item.title}</Text>
            <Text className="text-xs text-slate-500 mt-0.5">著者: {item.author}</Text>
            {/* ステータスバッジ。色はstatusに応じてクラスを切り替える */}
            <View className={`self-start mt-2 px-2.5 py-0.5 rounded-full
${getStatusClass(item.status)}`}>
                <Text className="text-xs font-semibold text-slate-700">{item.status}
            </Text>
            </View>
        </Pressable>
    )}
    />

    {/* 追加ボタン (FAB) */}
    <Pressable
        className="absolute right-5 bottom-8 w-14 h-14 rounded-full bg-blue-800
justify-center items-center shadow-lg"
        onPress={() => router.push("/new")}
    >
        <Text className="text-white text-3xl leading-8">+</Text>
    </Pressable>
</View>
);
}

// ステータスごとの背景色クラスを返す (第7章のgetStatusStyleのNativeWind版)
function getStatusClass(status: string) {
    if (status === "読了") return "bg-green-100";

```

```
if (status === "読書中") return "bg-blue-100";
return "bg-amber-100";
}
```

`className` の中で ``${ }`` を使う: `className={`... ${getStatusClass(item.status)}`}` のように、テンプレートリテラルで動的にクラスを足せます。ステータスによってバッジの背景色を変える、といった「条件で見た目を変える」処理に便利です。

contentContainerClassName: `FlatList` の内側のコンテナに対するクラス指定です。`StyleSheet` 版の `contentContainerStyle` に対応します。NativeWindは主要な部品でこうした `~ClassName` を用意しています。

StyleSheetと混在してOK: すべてを一度に書き換える必要はありません。NativeWindと `StyleSheet` は同じプロジェクトで共存できます。新しい画面はNativeWind、既存はそのまま、でも問題ありません。少しずつ慣れていきましょう。

▼ コードを1つずつ分解して解説

ここでは、見た目を決めている `className` の塊を中心に取り上げます。ロジック（`useState` や `useMemo` など）は第7・8章で解説済みなので、本節では「クラス名が何を意味するか」に絞って読み解きます。

解説1: 読み込み中の中央配置（ローディング画面）

```
<View className="flex-1 justify-center items-center">
  <ActivityIndicator size="large" color="#1e40af" />
</View>
```

- データ取得中（`loading` が `true`）のときに表示する、くるくる回る読み込み表示の枠です。
- `flex-1` ... この `<View>` を画面いっぱいに広げます。
- `justify-center`（縦中央）と `items-center`（横中央）の組み合わせで、`ActivityIndicator`（読み込みアイコン）が画面のど真ん中に来ます。
- 見た目の指定が `className` の3語だけで済み、`StyleSheet.create` のブロックが不要になっている点に注目してください。

用語: `ActivityIndicator` ... React Native標準の「処理中を示すくるくるアイコン」。`size` で大きさ、`color` で色を指定します。

解説2: 一覧画面の外枠（背景色付きの全画面コンテナ）

```
<View className="flex-1 bg-slate-50">
```

- 一覧画面全体を包む一番外側の枠です。
- `flex-1` ... 画面いっぱいに広がります。
- `bg-slate-50` ... 背景を「ごく薄いグレー」にします。`50` は一番薄い濃さで、白に近い上品な背景色になり、上に乗る白いカードが少し浮いて見えます。

用語: `bg-slate-50` ... `bg-`（背景色） + `slate`（青みのあるグレー） + `50`（最も薄い濃さ）。背景をほんのり色づけたいときの定番です。

解説3: 検索ボックス（余白つき枠と入力欄のスタイル）

```
<View className="p-4 pb-0">
  <TextInput
    className="bg-white border border-slate-200 rounded-lg px-4 py-2.5 text-sm"
    placeholder="🔍 タイトルや著者で検索..."
    value={keyword}
    onChangeText={setKeyword}
  />
</View>
```

- 外側の `<View className="p-4 pb-0">` ... `p-4` で四方に16pxの余白を付けつつ、`pb-0` で「下の余白だけ0」にしています（下にあるリストと間隔が空きすぎないようにする調整）。
- 入力欄 `<TextInput>` の `className` は5つの塊でできています。
- `bg-white` ... 背景を白に。
- `border border-slate-200` ... 「枠線あり」（`border`）かつ「枠線色は薄いグレー」（`border-slate-200`）。
- `rounded-lg` ... 角をやや丸くします。
- `px-4 py-2.5` ... 左右に16px・上下に10pxの内側余白。`px` は横方向、`py` は縦方向の余白です。
- `text-sm` ... 入力文字をやや小さめに。

用語: `px` / `py` / `pb` ... `p`（padding = 内側の余白）に方向を足した書き方。`x` = 左右、`y` = 上下、`b` = 下（`t` = 上、`l` = 左、`r` = 右）。数字は基本「4倍するとpx」（`p-4` = 16px、`py-2.5` = 10px）です。

解説4: リスト全体の余白とすき間 (`contentContainerClassName`)

```
<FlatList
  data={filteredBooks}
  keyExtractor={({item}) => item.id}
  contentContainerClassName="p-4 gap-2.5" // contentContainerStyleのNativeWind版
  ...
```

- `contentContainerClassName` は「`FlatList` の内側 (中身を並べるコンテナ) に当てるクラス」です。`StyleSheet` 版の `contentContainerStyle` に対応します。
- `p-4` ... リストの中身の四方に16pxの余白を付け、画面端に貼り付かないようにします。
- `gap-2.5` ... 並んだカード同士のすき間を10px空けます。`gap` を使うと、各カードに余白を付けなくても一定間隔で並びます。

用語: `gap` ... 並んだ要素同士の「あいだのすき間」をまとめて指定する書き方。1つ1つに `margin` を付けるより簡潔に等間隔を作れます。

解説5: 空のときのメッセージ (中央寄せ・薄色・上余白)

```
<Text className="text-center text-slate-400 mt-10 px-5">
  {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりません。"}
</Text>
```

- リストが空のとき (`ListEmptyComponent`) に表示する案内文です。
- `text-center` ... 文字を中央寄せにします。
- `text-slate-400` ... 文字色を「薄めのグレー」にして、目立ちすぎない控えめな案内にします。
- `mt-10` ... 上に40pxの余白 (`mt` = margin-top) を空け、画面の少し下に表示します。
- `px-5` ... 左右に20pxの余白を付け、長文でも画面端に触れないようにします。

用語: `mt` ... `m` (margin = 外側の余白) の上方向 (top) 。要素の「外側」に間隔を空けたいときは `m` 系、「内側」に空けたいときは `p` 系を使います。

解説6: 本のカード（白背景・角丸・枠線）

```
<Pressable
  className="bg-white rounded-xl p-3.5 border border-slate-200"
  onPress={() => router.push(`/books/${item.id}`)}
>
  <Text className="text-base font-bold text-slate-800">{item.title}</Text>
  <Text className="text-xs text-slate-500 mt-0.5">著者: {item.author}</Text>
```

- 1冊分のカードを表す `<Pressable>`（押せる領域）のスタイルです。
- `bg-white` ... カード背景を白に（薄グレー背景の上で浮いて見えます）。
- `rounded-xl` ... 角を大きめに丸くします（`lg` よりさらに丸い）。
- `p-3.5` ... 四方に14pxの内側余白を付けます。
- `border border-slate-200` ... 薄いグレーの枠線を付けて輪郭をはっきりさせます。
- カード内のテキスト。
- タイトルは `text-base`（標準サイズ） + `font-bold`（太字） + `text-slate-800`（濃いグレー）で見出しらしく。
- 著者名は `text-xs`（小さめ） + `text-slate-500`（中間グレー） + `mt-0.5`（上に2pxのわずかな余白）で、タイトルの下に控えめに添えます。

用語: `<Pressable>` ... タップに反応するReact Nativeの部品。 `onPress` に押されたときの処理を渡します。ここでは押すと詳細画面へ移動します。

解説7: ステータスバッジ（テンプレートリテラルで色を切り替える）

```
<View className={`self-start mt-2 px-2.5 py-0.5 rounded-full
  ${getStatusClass(item.status)}`} >
  <Text className="text-xs font-semibold text-slate-700">{item.status}</Text>
</View>
```

- 「読了／読書中／未読」などの状態を示す小さなバッジです。 `className` がテンプレートリテラル（``...``）になっている点が新しいところです。
- `self-start` ... バッジを「左端に合わせ、横幅を中身ぴったり」にします（横いっぱいには広がらないようにする指定）。
- `mt-2` ... 上に8pxの余白。
- `px-2.5 py-0.5` ... 左右10px・上下2pxの内側余白で、文字を小さな枠で囲みます。
- `rounded-full` ... 角を完全に丸めることで、カプセル型（ピル型）のバッジになります。
- `${getStatusClass(item.status)}` ... ここに状態ごとの背景色クラスが差し込まれます（次の解説8）。
- 中の文字は `text-xs`（小） + `font-semibold`（やや太字） + `text-slate-700`（やや濃いグレー）。

用語: テンプレートリテラル ... バッククォート ` ` で囲んだ文字列。中で `${ }` を使うと、変数や関数の戻り値を埋め込みます。固定のクラスに「条件で変わるクラス」を足すときに便利です。

解説8: 状態に応じて背景色クラスを返す関数（`getStatusClass`）

```
function getStatusClass(status: string) {
  if (status === "読了") return "bg-green-100";
  if (status === "読書中") return "bg-blue-100";
  return "bg-amber-100";
}
```

- バッジの背景色クラスだけを、状態の文字列に応じて返す小さな関数です（第7章の `getStatusStyle` の NativeWind版）。
- `status` が `"読了"` なら `bg-green-100`（薄い緑）、`"読書中"` なら `bg-blue-100`（薄い青）を返します。
- どちらにも当てはまらない場合（未読など）は、最後の `return "bg-amber-100"`（薄い黄色）が使われます。
- いずれも濃さ `100` の淡い背景色なので、上に乗る濃いめの文字（`text-slate-700`）が読みやすくなります。

用語: `bg-green-100` など ... 背景色を「薄い色（濃さ100）」で当てるクラス。バッジやタグのように「うっすら色を付けて種類を区別したい」場面でよく使います。

解説9: 追加ボタン（画面右下に浮かぶ丸ボタン＝FAB）

```
<Pressable
  className="absolute right-5 bottom-8 w-14 h-14 rounded-full bg-blue-800 justify-
center items-center shadow-lg"
  onPress={() => router.push("/new")}
>
  <Text className="text-white text-3xl leading-8">+</Text>
</Pressable>
```

- 画面の右下に常に浮いている丸い追加ボタン（FAB＝Floating Action Button）のスタイルです。クラスが多いので塊ごとに見ます。
- `absolute right-5 bottom-8` ... 位置を固定（`absolute`）し、右から20px・下から32pxの位置に置きます。これでスクロールしても右下に浮き続けます。
- `w-14 h-14` ... 幅と高さを56pxにして正方形に（`w`＝幅, `h`＝高さ）。
- `rounded-full` ... 角を完全に丸めて真円にします。
- `bg-blue-800` ... 背景を濃い青に。

- `justify-center items-center` ... 中の「+」を上下左右の中央に置きます。
- `shadow-lg` ... 大きめの影を付けて、ボタンが浮いて見えるようにします。
- 中の `<Text className="text-white text-3xl leading-8">` ... 「+」を白（`text-white`）・大きい文字（`text-3xl`）にし、`leading-8`（行の高さ32px）で上下位置を微調整しています。

用語: FAB（Floating Action Button）... 画面に固定で浮かぶ丸い操作ボタン。`absolute` で位置を固定し、`rounded-full` で円形にするのが定番の作り方です。

5. アプリ全体の仕上げ

機能と見た目が整ったら、公開前に「アプリらしさ」を高める仕上げをしておくことで完成度が上がります。

5.1 アプリ名・アイコン・スプラッシュ画面

`app.json`（第1章で見た設定ファイル）で、アプリの基本情報を設定します。

▼このコードがやること（先に日本語で）：アプリの名前・アイコン・起動画面（スプラッシュ）などの基本情報を `app.json` にまとめて設定します。`name` はホーム画面に出るアプリ名、`icon` はアイコン画像、`splash` は起動時に一瞬出る画面の指定です。公開前の「アプリらしさ」を決める大事な設定なので、どの項目が何を表すかを各行のコメントで押さえておきましょう。

```
{
  "expo": {
    "name": "書籍管理",           // ホーム画面に表示されるアプリ名
    "slug": "my-books-app",      // プロジェクトの識別名（半角英数）
    "version": "1.0.0",          // アプリのバージョン
    "icon": "./assets/icon.png", // アプリアイコン（1024x1024の正方形画像）
    "splash": {                 // 起動時に一瞬出るスプラッシュ画面
      "image": "./assets/splash.png", // スプラッシュ画像
      "backgroundColor": "#1e40af"   // その背景色
    },
    "ios": { "supportsTablet": true }, // iPad対応
    "android": { "package": "com.example.mybooksapp" } // Androidの識別子（後で変更）
  }
}
```

アイコンとスプラッシュ画像: `assets` フォルダにExpoのデフォルト画像が入っています。まずはそのままでも公開できます。オリジナル画像に差し替えると一気に「自分のアプリ」感が出ます。アイコンは1024×1024ピクセルの正方形PNGを用意しましょう。第10章の公開前に整えるのがおすすめです。

▼ コードを1つずつ分解して解説

解説1: 設定全体の入れ物 ("expo")

```
{
  "expo": {
    "name": "書籍管理",
    "slug": "my-books-app",
    "version": "1.0.0"
  }
}
```

- `app.json` はExpoアプリの設定をまとめたファイルで、全体が `{ "expo": { ... } }` という形になっています。"expo" の中に各設定を並べます。
- `name` ... ホーム画面に表示されるアプリ名です。日本語でもOK。
- `slug` ... プロジェクトの識別名で、半角英数で付けます (URLなどに使われる内部用の名前)。
- `version` ... アプリのバージョン番号。更新のたびに上げていきます。

用語: JSON (ジェイソン) ... `{ "キー": 値 }` の形で設定やデータを書く形式。値は文字列なら必ず `"..."` で囲み、項目はカンマ `,` で区切ります。

解説2: アイコンと起動画面 (`icon` / `splash`)

```
"icon": "./assets/icon.png",
"splash": {
  "image": "./assets/splash.png",
  "backgroundColor": "#1e40af"
}
```

- `icon` ... ホーム画面に出るアプリアイコンの画像ファイルを指定します。1024×1024の正方形PNGが基本です。
- `splash` ... アプリ起動時に一瞬出る画面 (スプラッシュ) の設定をまとめたものです。
- `image` ... スプラッシュに表示する画像。
- `backgroundColor` ... その背景色。 `#1e40af` は濃い青を表すカラーコードです。
- これらを設定すると、公開前の「自分のアプリらしさ」がぐっと高まります。

用語: カラーコード (`#1e40af`) ... 色を6桁の英数字で表す書き方。 `#` のあとに赤・緑・青の強さを並べたもので、 `#1e40af` は濃い青になります。

```
"ios": { "supportsTablet": true },
"android": { "package": "com.example.mybooksapp" }
```

- `ios` と `android` は、それぞれのOS専用の設定をまとめる場所です。
- `ios.supportsTablet: true` ... iPad（タブレット）に対応するという指定です。
- `android.package` ... Androidアプリを世界で1つに識別するための文字列です。 `com.会社名.アプリ名` のような形式で書き、公開前に自分用の値へ変更します。

用語: パッケージ名 (`com.example.mybooksapp`) ... Androidでアプリを一意に識別するID。他のアプリと重複しないよう、自分が持つドメインを逆順にした形などで付けるのが慣習です。

5.2 ダークモード対応（発展・任意）

NativeWindは `dark:` というプレフィックスでダークモード（暗い配色）対応も簡単にできます。

```
// dark: を付けると、端末がダークモードのときだけそのクラスが適用される
<View className="bg-white dark:bg-slate-900">
  <Text className="text-slate-800 dark:text-slate-100">ダークモード対応のテキスト</Text>
</View>
```

任意の機能です: ダークモード対応は必須ではありません。「こんなこともできる」という紹介です。余裕があれば挑戦してみてください。

6. この章のまとめ

- スタイリングの選択肢 (StyleSheet / NativeWind / Tamagui / UIキット) を比較し、目的別の選び方を理解した
- NativeWind を導入 (インストール → `tailwind.config.js` → `global.css` → Babel/Metro設定 → `clear` 再起動)
- `style={...}` の代わりに `className="..."` でクラスを並べて見た目を指定する書き方を学んだ
- 一覧画面をNativeWindで書き直し、コードが短くなることを体感した
- `app.json` でアプリ名・アイコン・スプラッシュを設定し、公開の準備を整えた

次の章へ: アプリが完成しました！いよいよ第10章で、このアプリを App Store と Google Play に公開します。世界中の人があなたのアプリを使えるようになります。